

Java Fundamentals - Exercises

Java Introduction & Features

• What is Java and who developed it?

Java is a high-level, object-oriented programming language and it is used to build software and applications.

It was developed by James Gosling at Sun Microsystems in 1995.

• List any 5 features of Java.

1) Platform Independent: write once, run anywhere using JVM.

2) Object-oriented: Everything is treated as an object.

3) Secure: provided features like bytecode verification and no pointer access.

4) Multithreaded: can perform multiple tasks at same time.

5) Robust: Strong error handling and memory management make Java reliable.

• Why is Java called Platform-Independent?

Because Java code can run on any operating system or device that has a Java Virtual Machine.

Main Java → byte code → This byte code
(after compilation)

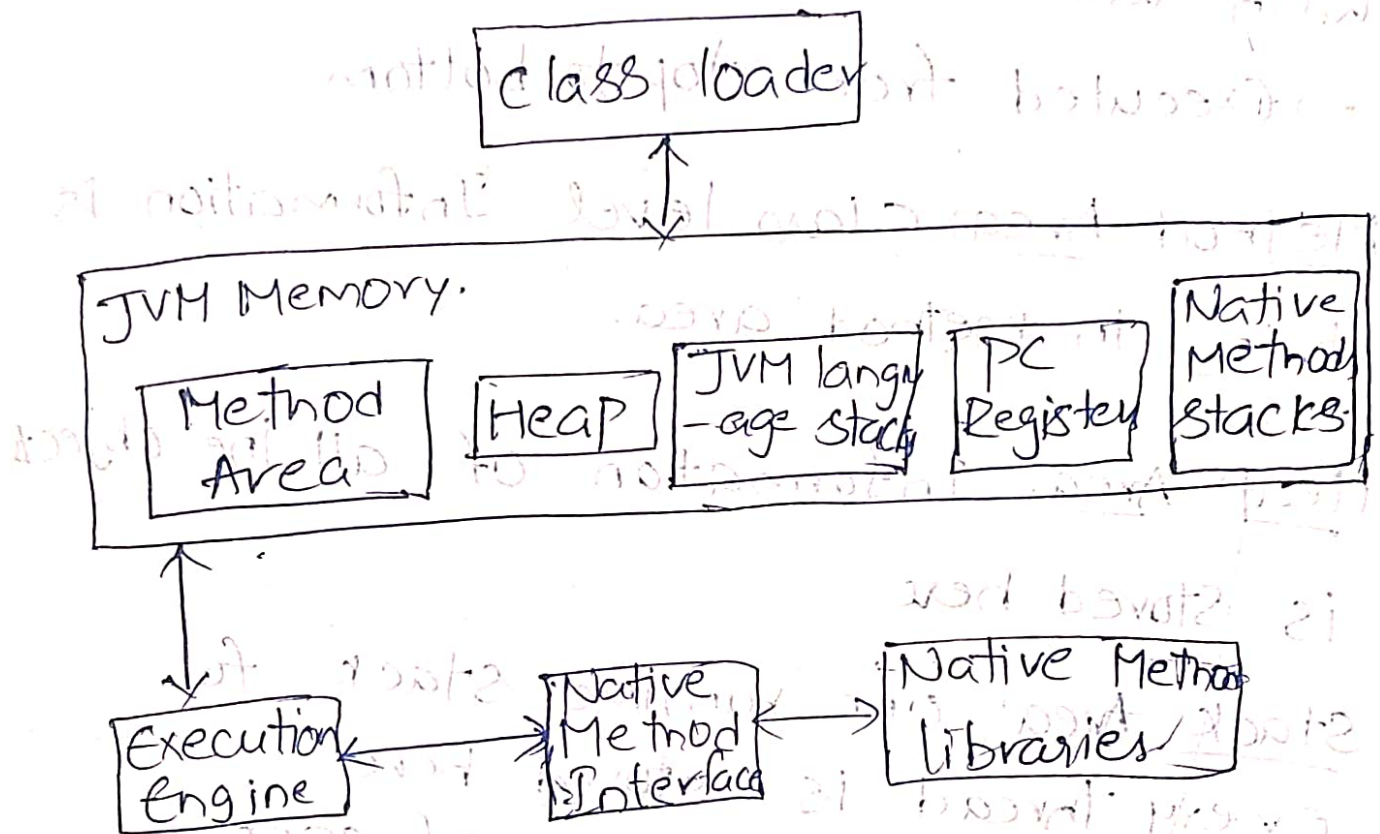
Run in any Platform if JVM exists in it.

JVM, JRE, JDK - Architecture

• Differentiate between JVM, JRE and JDK

Java Virtual Machine	Java Runtime Environment	Java Development Kit
converts byte code to Machine specific code	JVM, Other Clases.	JRE, Java Compiles
→ It Automatically Includes with JRE Installation.	→ It is used for executing programs without development or compilation.	→ For development, compilation and execution.
→ Highly platform dependent.	→ platform dependent	→ Platform dependent.

• Draw and Explain the architecture of JVM.



• Class loader is responsible for 3 activities

- 1) Loading - Reads class files
- 2) Linking - Generate binary data.
- 3) Initialization

Ensures correctness of class files:

- check for format
- java.lang.VerifyError if verification fails.

Initialization: static variables are assigned with values.

→ Executed from top to bottom.

Method Area: Class level Information is stored in method area.

Heap Area: Information of all the objects is stored here.

Stack Area: One runtime stack for every thread is stored here.

→ It stores the information of each Method call.

PC Register: stores address of current execution instruction of thread.

Native Method Stacks: Separate native stack for every thread.

Execution Engine: It is used to execute byte code. There are 3 parts in it:

- 1) Interpreter
- 2) Just in time Compiler
- 3) Garbage collection

Native Method Interface:

It contains native Method libraries.

Native Method libraries:

collection of the java libraries which are required by the Execution Engine.

- What role does the JIT compiler play?
- It Increase the Efficiency of Interpreter and it provides direct native code to avoid re Interpretation

Java Program Structure

- write a basic Java program

Amprint class Main

```
{  
    public static void main(String[] args)
```

```
{
```

```
    System.out.println("Hello, Java!");
```

```
}
```

```
}
```


• Explain each part of Java Program.

Ans.

• Public class Main - class Definition

• Public static void main (String[] args)

- This is Entry point of the program.

It is where the program execution starts.

• System.out.println(""); - prints the text to the console.

• What is the role of main Method.

Ans. It is the starting point of any standalone Java application.

Public - Access Modifier. It is used by JVM to access it from outside of the class.

Static - No need to create object to call it.

void - It doesn't return any type

main - The name of the method.
String[] args - Accept command line arguments.

Keywords & Identifiers:

• Define keywords with example.
Ans. These are reserved words in Java and they have special meaning to the compiler.

• we can't use them as Method or variable name.

• List 10 keywords in Java.

- Ans:
- | | |
|-----------|------------|
| 1) class | 6) if |
| 2) public | 7) else |
| 3) static | 8) new |
| 4) void | 9) return |
| 5) int | 10) break. |

- What are the identifiers and what rules apply to them.

Identifiers are used for identity.

- classes
 - variables
 - Methods.
 - objects
 - Interfaces.
- Rules: can contain letters (A-Z, a-z)

1. digits (0-9), \$ and _.

• can't be Java keyword.

• Java is case-sensitive

Variables and Data types

- What is primitive and reference data types in Java?

Ans) These are used to define what kind of data a variable can store.

- stores in stack memory.

Primitive Data types:

These are basic built-in types.

- 1) byte,
- 2) short
- 3) int
- 4) long
- 5) float
- 6) double
- 7) char
- 8) boolean.

Reference Data types:

- These refers to objects in memory.
- Stores in heap Memory.
- They don't store the value directly. They store the memory address.

- 1) String
- 2) Arrays
- 3) Objects
- 4) Interfaces.

Declare variables of all primitive data types.

```
byte b; byte1 = 100;
```

```
short s; short1 = 200;
```

```
int i; int1 = 300;
```

long long1 = 40000;

float float1 = 5.10f;

double double1 = 6.66;

char char1 = 'A';

boolean boolean1 = true;

write a program to display student name,
Age and grade.

Ans

```
public class student1 {  
    public static void main (String[] args)  
    {  
        String name = "Aditya"  
        int age = 21;  
        char grade = 'A';  
        System.out.println("Student name:"  
                             + name);  
        System.out.println("Student age:"  
                             + age);  
        System.out.println("Student grade:"  
                             + grade);  
    }  
}
```

Type Casting:

- Differentiate between Implicit and explicit

<u>Implicit casting</u>	<u>Explicit casting</u>
• widening conversion • It automatically performed by Java Compiler • No data loss • Smaller type $\rightarrow$ larger type ex: <code>int x = 10;</code> <code>long y = x;</code>	• Narrowing conversion • Programmer needs to cast with operator • May lead to data loss • Larger type $\rightarrow$ smaller type ex: <code>double d = 100.111;</code> <code>int x = (int) d;</code>

- write a program to demonstrate both types of casting.

```
public class Main
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
    int a = 10;
```

```
    long b = a;
```

```
    double c = 6.66;
```

```
    int d = (int) c;
```

```
    System.out.println("long value: " + b);
```

```
    System.out.println("double to int
```

```
value: " + d);
```

```
}
```

```
}
```

• what happens when narrowing conversion fails?

It gives the garbage & unexpected values

```
ex: int a = 130;
    byte b = (byte) a;
    S.O.P(b);
```

output: -126

Operators

• Explain the difference between `==` and `=` operators

Ans `==` compares two values and returns boolean value

`=` Assigns a value to the variable

• write a program using arithmetic and relational operators.

```
public class Main
{
    public static void main (String[] args)
    {
        int a = 100;
        int b = 50;

        System.out.println("Addition: " + (a+b));
        System.out.println("Substraction: " + (a-b));
        System.out.println("Multiplication: " + (a*b));
        System.out.println("Division: " + (a/b));
        System.out.println("Modulus Division: " + (a%b));
        System.out.println("a==b: " + (a==b));
        System.out.println("a>b: " + (a>b));
        System.out.println("a<b: " + (a<b));
        System.out.println("a>=b: " + (a>=b));
        System.out.println("a<=b: " + (a<=b));
        System.out.println("a!=b: " + (a!=b));
    }
}
```


output:

Addition: 150

Subtraction: 50

Multiplication: 5000

Division: 2

Modulus Division: 0

$a = b$: false

$a > b$: true

$a < b$: false

$a \geq b$: ~~false~~ true

$a \leq b$: false

$a != b$: true

• List & explain logical operators.

- 1) $\&\&$ - Returns true if both conditions are true
- 2) $\|\|$ - Returns true if either one of them are true.
- 3) $!$ - It inverts the condition

Control Flow

- write a program to find the largest of three numbers using if-else.

```
import java.util.*;  
public class Main {
```

```
{  
    public static void main (String[] args)
```

```
{  
    Scanner sc = new Scanner (System.in);
```

```
    int a = sc.nextInt();
```

```
    int b = sc.nextInt();
```

```
    int c = sc.nextInt();
```

```
    if (a >= b && b >= c)
```

```
{
```

```
        System.out.println("Largest number  
        is: " + a);
```

```
}
```

```
    else if (b >= a && b >= c)
```

```
{
```

```
        System.out.println("Largest number  
        is: " + b);
```

```
}
```

```
    else
```

```
{
```

```
        System.out.println("Largest number
```

```
}
```

```
is: " + c);
```

• Demonstrate the use of switch case.

It is used to select one option from multiple choices based on a variable's value.

```
switch (expression)
{
    case value1:
        break;
    case value2:
        break;
    . . .
    default:
        . . .
}
```


• Write a loop that prints numbers from 10 to 1.

Ans) Public class Main

{

Public static void main (String[] args)

{

int i;

for (i=10; i>0; i--)

{

System.out.println (" " + i);

}

System.out.print();

}

}

Arrays: Array is the collection of elements of the same data type, stored in single variable and stored in contiguous memory location.

→ The length of the arrays are fixed.

Syntax:

```
Datatype [Name of Array] = new Datatype[size]
```

```
{  
    // Declaration  
    Datatype [Name] = new Datatype[size];  
}
```

```
{  
    // Declaration  
    Datatype [Name] = new Datatype[size];  
}
```

```
{  
    // Declaration  
    Datatype [Name] = new Datatype[size];  
}
```

Example: To create an array of 10 elements.

- write a programme to find the sum of elements in an array.

```
public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int sum = 0;
        int size = sc.nextInt();
        int[] a = new int[size];
        for (int i = 0; i < size; i++)
        {
            a[i] = sc.nextInt();
        }
        for (int i = 0; i < size; i++)
        {
            sum += a[i];
        }
        System.out.println("Sum of elements  
in the array is: " + sum);
    }
}
```


• declare and initialize a 2D array and print its elements.

```
public class Main
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
    int[][] a = new { { 1, 2, 3 }, { 4, 5, 6 },  
                        { 7, 8, 9 } };
```

```
    for (int i = 0; i < a.length; i++)
```

```
    {
```

```
        for (int j = 0; j < a[i].length; j++)
```

```
        {
```

```
            System.out.print (a[i][j] + " ");
```

```
        }
```

```
        System.out.println();
```

```
    }
```

```
}
```

```
}
```